Extended Abstract

Motivation Mathematical reasoning is a core component of assessing human and, thereby, AI intelligence. However, large language models, despite their success in natural language processing and instruction following, struggle with precise arithmetic and reliable factual retrieval. These limitations often result in hallucinated or incorrect outputs, particularly in math domains. While tool-augmented approaches, such as Toolformer and Search-R1, have demonstrated improved robustness in general tasks by granting models access to external APIs or search engines, their efficacy in mathematical reasoning remains underexplored.

Method This work addresses the gap in performance in math domains by fine-tuning the Qwen 2.5 0.5B Base model on the WarmStart and Countdown datasets using supervised learning and reinforcement learning with RLOO, and subsequently enhancing the model's capabilities through synthetic tool-augmented data that simulate calculator usage.

Implementation First, we start with supervised finetuning of the Qwen 2.5 0.5B Base model on the WarmStart dataset using a training set of 1000 examples and a validation set consisting of 200 examples. After training, we evaluated our model on the test split in the Countdown dataset and held-out prompts on the leaderboard. Next, we implemented RLOO on the Countdown dataset, where we evaluated the model based on the average rewards on the test set. Finally, we constructed a synthetic dataset using our WarmStart-SFT model to generate a set of incorrect and correct responses per prompt that we then use to provide the LLM examples of how to use the calculator. Finally, we use this synthetic dataset to perform SFT on our RLOO model. We use cross-entropy loss calculated on a per-token level and evaluated the model based on per-token accuracy during the training of SFT. We masked out prompts and system messages during loss calculation.

Results For WarmStart, our initial SFT model achieved an average 0.5117 cross entropy loss and 84.39% token-wise accuracy for training and evaluation. We evaluated our model using VLLM on Countdown (Gandhi et al. (2024)) with 1000 random responses and our model achieved a score of 0.3088 using the rule-based reward function from Gandhi et al. (2025). The RLOO model achieved the highest average reward of 0.5163 on the validation set during training and 0.3498 on the training set at the same step. Our RLOO model significantly outperformed the SFT model on both the test split of the Countdown dataset and the held-out leaderboard prompts, achieving an average score of 0.4204 on the test split and 0.3909 on the leaderboard prompts. The extension model trained on the synthetic tool-augmented dataset achieved 0.166 cross entropy loss and 92% token-wise accuracy for evaluation. When limited to three rounds of calculator-based answer verification, the model outperformed the RLOO baseline, achieving an average score of 0.5324 on the test split of the Countdown dataset and 0.4704 on the held-out leaderboard prompts. When we allowed the model to call the calculator up to ten times for answer verification, it achieved the highest score, 0.6074, on the test split and the highest score, 0.7117, on the leaderboard held-out prompts.

Discussion Our findings show that both fine-tuning and interactive tool use significantly improve mathematical reasoning performance, motivating further exploration and usage of tool-augmented learning for more reliable LLMs especially in mathematically intensive applications. In addition, our method can be extended to other datasets and incorporate the use of other tools. For example, it can be extended to search to identify sources for the answer the LLM is giving. Another tool that the LLM can incorporate is the weather app or the news app to give the LLM more reliability regarding current events.

Conclusion Our results suggest that incorporating tool usage is a promising strategy for enhancing the accuracy and reliability of large language models. This approach supports the idea that tools can play a key role in improving LLM reasoning.

Effective Tool Use: RL Fine-tuning of Language Models

Ziwei Chen
Department of Computer Science
Stanford University
ziwei75@stanford.edu

Arpita Singhal
Department of Computer Science
Stanford University
arpitas@stanford.edu

Abstract

Large language models have demonstrated impressive performance on a variety of natural language processing tasks and the ability to remarkably generate coherent responses and solve complex tasks. In this paper, we demonstrate that LLMs can learn to use external tools, specifically the calculator, to help improve their accuracy and performance since they struggle with basic factual tasks like arithmetic. We perform SFT on the Qwen 2.5 0.5B Base model on the WarmStart dataset, train with RLOO on the Countdown dataset, and then perform another round of SFT with a synthetic dataset that we generated. Our model achieves substantially improved performance when tested on the Countdown dataset as we provide more supervision and tool usage. Our work highlights the importance of tool usage in developing more reliable and accurate LLMs for mathematical reasoning.

1 Introduction

Mathematical reasoning constitutes a core aspect of cognitive ability and was regularly used as part of an assessment for human intelligence. Large language models (LLMs) have demonstrated proficiency in generating coherent responses and following complex instructions. However, increasing research indicates that LLMs struggled on performing precise mathematical operation and encounter obstacle on solving problems in mathematical domain Ahn et al. (2024). In addition, it has been shown that although LLMs are efficient at storing knowledge in their networks, they sometimes face difficulties on retrieving factual information and would hallucinate unreliable results (Komeili et al. (2022), Joshua Maynez (2020)). In mathematical reasoning tasks, LLMs may make errors in basic arithmetic operations, leading to incorrect results in downstream problem-solving. Recent research has shown that giving LLMs access to external tools can significantly improve their ability to retrieve up-to-date information and produce more reliable results (Schick et al. (2023), Jin et al. (2025)). For instruction-following tasks where LLMs generate responses or perform actions based on natural language prompts, incorporating tools such as search engines and weather apps has been shown to enhance model robustness across various benchmarks. However, the impact of incorporating tool use has not yet been thoroughly evaluated in the context of mathematical reasoning tasks. In this project, we first finetuned the large language model (LLM), Qwen 2.5 0.5B Base, on performing mathematical reasoning for the Countdown dataset Gandhi et al. (2024). We first implemented Supervise Fine Tuning (SFT) on the Warmstart dataset Gandhi et al. (2025) and then used Online Policy gradient, specifically RLOO (Ahmadian et al. (2024),) to train our model on the Countdown dataset. Next, we used our SFT model to generate a synthetic dataset illustrating the use of a calculator for answer verification. We then fine-tuned the model on this synthetic, tool-augmented dataset and evaluated its performance by providing access to a calculator during inference.

Our results demonstrated that Supervised Fine-Tuning (SFT) can be highly effective in aligning large language models (LLMs) to specific tasks. RLOO is efficient at improving model performance when clear rewards are defined for the task. We also showed that enabling tool usage can boost our

model performance by allowing it to verify and correct its mistakes and the performance continued to improve when we increased the number of interaction rounds between the model and the tools. These results highlight the importance of finetuning, RL optimization and tool augmentation in building more reliable LLMs for mathematical reasoning.

2 Related Work

LLMs have achieved impressive results on various benchmark in different domains, including coding, instruction following, and mathematical reasoning (DeepSeek-AI et al. (2025), Ahn et al. (2024)). However, growing evidence suggests that while LLMs excel at understanding and generating human-like text, they remain prone to hallucinating results and generating false information (Komeili et al. (2022)). Incorporating the usage of tools to provide LLMs with up-to-date knowledge has been shown to be effective on reducing hallucination and increase model robustness. For instance, Jin et al. (2025) developed Search-R1 which provides LLMs access to search engines with real-time retrieval and has been shown to improve the performance on seven question-answering datasets over various baselines. Schick et al. (2023) proposed Toolformer which was trained to call various external tools and APIs and has achieved substantial improvement on zero-shot performance on various tasks.

However, one limitation of Toolformer is that the model only evaluates API calls and incorporates the results into its final prediction without using the responses to iteratively refine or modify its output. In contrast, Search-R1 supports multi-turn search interactions, allowing the model to improve its responses based on feedback from the tools. Nevertheless, Search-R1 has only been benchmarked on question-answering datasets with the usage of search engines and has not been evaluated on mathematical reasoning tasks. Thus, in this project, we aim to extend Search-R1 by leveraging similar multi-turn tool interactions, with a specific focus on evaluating the model's performance on mathematical reasoning tasks by providing the LLM access to a calculator.

3 Method

3.1 Default Implementation

The task in both datasets, including WarmStart and Countdown, involves using a given set of numbers to reach a target number by applying basic arithmetic operations (addition, subtraction, multiplication, and division). For example: Using the numbers [95, 36, 32], create an equation that equals 91 using basic arithmetic operations. The Countdown (Gandhi et al. (2024)) dataset only has the set of numbers to use and the target number while the WarmStart (Gandhi et al. (2025)) dataset also provides the solution and detailed reasoning on how to arrive at a particular solution.

3.1.1 Supervised Fine-Tuning using WarmStart

To align our model to the mathematical reasoning task, we first implemented Supervised Fine-Tuning (SFT) via behavior cloning to warm start our model, Qwen 2.5 0.5B Base, using the WarmStart dataset. We used the cross entropy (CE) loss calculated on per-token level and evaluated the model based on per-token accuracy during the training of SFT. We used a training dataset with 1000 examples and 200 examples in the validation set. After training, we evaluated our model on the test split in the countdown dataset and held-out prompts on the leaderboard.

3.1.2 RLOO on Countdown

Next, we further improved our model's performance on mathematical reasoning by implementing RLOO (Ahmadian et al. (2024)) on the Countdown dataset. RLOO is a REINFORCE-based policy gradient estimator. It reduces the variance in advantage estimation by sampling K responses from the model for each prompt during training and uses the average reward of other samples as a baseline to compute the advantage for each response. During training, we evaluated our model based on the average rewards on the test set, and the final model was evaluated on both the test split in the countdown dataset and held-out prompts on the leaderboard.

3.2 Extension Implementation

3.2.1 Synthetic Data Generation

For the extension, we focused on implementing tool usage, inspired by the work in Schick et al. (2023). Specifically, we focused on adding the use of the calculator for the model to verify its responses. We generated a synthetic dataset using the outputs from the Qwen 2.5 0.5 Base model, trained with SFT on the WarmStart dataset. We generated 8 responses for each input prompt, covering 20,000 examples from the training dataset and 5,000 from the test dataset. We used the rule-based reward function from Gandhi et al. (2025) to identify responses that were (1) correctly-formatted but incorrect or (2) correctly-formatted and also accurate. We removed any responses that did not have either of the responses.

We devised the following prompt for each response by editing the initial prompt from WarmStart:

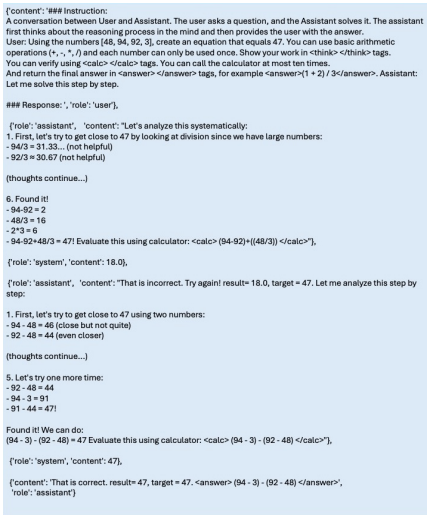
> ### **Instruction**:A conversation between User and Assistant. The user asks a question, and the Assistant solves it. The assistant first thinks about the reasoning process in the mind and then provides the user with the answer. User: Using the numbers [*input_1*, *input_2*, *input_3*], create an equation that equals *target_number*. You can use basic arithmetic operations (+, -, *, /) and each number can only be used once. Show your work in <think> </think> tags. You can verify using <calc> </calc> tags. And return the final answer in <answer> </answer> tags, for example <answer>(1 + 2) / 3</answer>. **Assistant**: Let me solve this step by step. ### **Response**:

We then inserted a minimum of one to a maximum of three iterations of incorrect *<think></think>*, *<calc></calc>* responses to demonstrate to the model how to use the calculator to verify its responses. We used a *System* output to verify the responses in the *<calc></calc>*. Finally, we ended each example with a correct response verified by the *System* and outputted the answer in *<answer></answer>*.

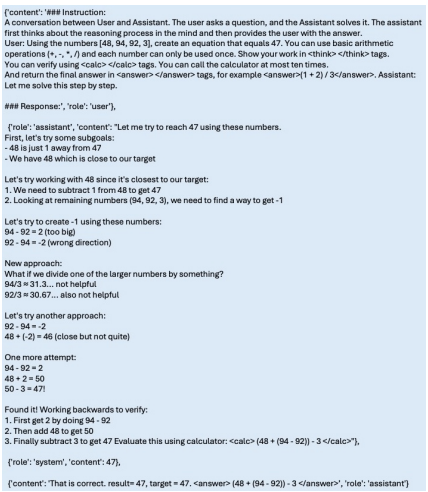
We also generated a dataset that only consisted of correct *<think></think>*, *<calc></calc>* responses because we wanted the model to focus on learning from correct examples.

After filtering prompts with no correct answers, we had a total of 11,909 examples in the train dataset with both incorrect and correct thought processes and answers. There were a total of 17,111 examples in the train dataset with only correct thought processes and answers. Finally, we had a total of 1,756 examples in the validation dataset with both incorrect and correct thought processes and answers.

Examples of the generated synthetic datasets can be found in Figure 1a and Figure 1b.



(a) Only correct thought processes.



(b) Both correct and incorrect thought processes.

Figure 1: Synthetic Data Generation Examples

3.2.2 Supervised Fine-Tuning using the Generated Synthetic Dataset

To align our model to the calculator reasoning task, we implemented Supervised Fine-Tuning (SFT) using the RLOO model on the synthetic tool-augmented dataset. We used the cross entropy (CE) loss calculated on per-token level and evaluated the model based on per-token accuracy during the training of SFT. We trained the model in a multi-turn interaction setting by providing the full assistant and system interactions as input, while masking out prompts and system messages during loss calculation.

3.3 Evaluation

We randomly selected 1000 prompts from the countdown test set and evaluated the performance of our model on them using the rule-based reward function from Gandhi et al. (2025). A response receives a score of 0.1 if it is correctly formatted, and a full score of 1.0 if it is both correctly formatted and evaluates to the target value.

We also evaluated our models using the held-out prompts from the leaderboard by submitting the generated responses from each model to the leaderboard.

4 Experimental Setup

For SFT on the Warmstart dataset, each example was truncated and padded to 1024 tokens. We used cross-entropy loss and finetuned our model using the Adam optimizer with a learning rate scheduler, which starts at 0 and warms up linearly to 1e-4 for the first 100 steps and then decays linearly to 0 over the remaining steps. We experimented with different learning rates and finalized this as the final learning rate setup as it had the highest evaluation accuracy. We used a batch size of 2 and trained for 10 epochs.

For RLOO on the Countdown dataset, we used a batch size of 4 prompts and sampled 4 responses per prompt, with each response capped at 512 tokens. The model was evaluated every 50 training steps based on the average reward over 40 examples randomly sampled from evaluation set. We used a learning rate of 3e-6 and applied gradient accumulation every 4 steps. To stabilize training, we computed the KL divergence between our model and the SFT model, using a KL coefficient of 0.01 to prevent the policy from drifting too far from our SFT baseline.

For SFT on the synthetic dataset, each example was truncated and padded to 2500 tokens as the maximum length of the example is around 2300 after tokenization. We used the RLOO model as the baseline and finetuned it on the synthetic dataset using a batch size of 8 for 10 epochs. We trained the model using the Adam optimizer using a learning rate of 2e-5 and weight decay of 0.01. The model was evaluated on the test set for every 200 steps and the model with the lowest evaluation loss was saved.

5 Results

5.1 SFT

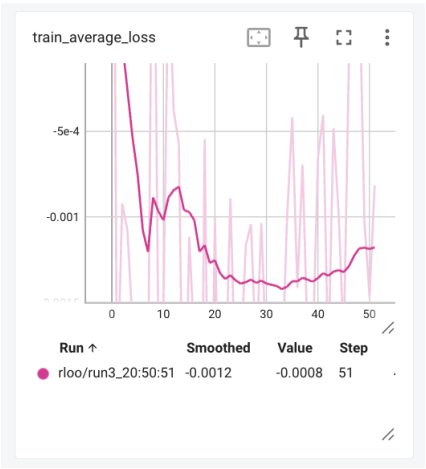
For WarmStart, the SFT model achieved an average 0.5117 cross entropy loss and 84.39% token-wise accuracy for training and evaluation. We evaluated our model using VLLM on Countdown (Gandhi et al. (2024)) with 1000 random responses and our model achieved a score of 0.3088 using the rule-based reward function from Gandhi et al. (2025). We also evaluated our SFT model by submitting it to the leaderboard, where it achieved an average score of 0.2800 on the held-out leaderboard prompts. This submission was made during the second round of the leaderboard, meaning the SFT model was evaluated on the same set of prompts as our RLOO and extension models.

5.2 RLOO

The RLOO model achieved the highest average reward of 0.5163 on the validation set during training, and 0.3498 on the training set at the same step. This discrepancy between training and validation may simply be due to stochastic variation in response sampling. The plot for the average loss and average rewards for training and evaluation were shown in Figure 2 and 3. Despite the stochasticity

in rewards and losses, we observe a general trend of increasing rewards and decreasing losses during RLOO, for both training and evaluation.

As shown in Table 1, our RLOO model significantly outperformed the SFT model on both the test split of the Countdown dataset and the held-out leaderboard prompts, achieving an average score of 0.4204 on the test split and 0.3909 on the leaderboard prompts.



(a) RLOO training loss vs. step

(b) RLOO evaluation loss vs. step

Figure 2: RLOO loss vs. step

(a) RLOO training rewards vs. step

(b) RLOO evaluation rewards vs. step

Figure 3: RLOO rewards vs. step

5.3 SFT on the synthetic tool-augmented dataset

The extension model trained on the synthetic tool-augmented dataset achieved 0.166 cross entropy loss and 92% token-wise accuracy for evaluation. We then evaluated our extension model by providing it a calculator during inference time. When limited to three rounds of calculator-based answer verification, the model outperformed the RLOO baseline, achieving an average score of 0.5324 on the test split of the Countdown dataset and 0.4704 on the held-out leaderboard prompts.

We also examined how the number of calculator calls allowed for answer verification affects the model's final performance. As expected, allowing more calculator calls improved evaluation performance on both the test split and the leaderboard prompts. As shown in table 1, when we allowed

Table 1: Performance Comparison

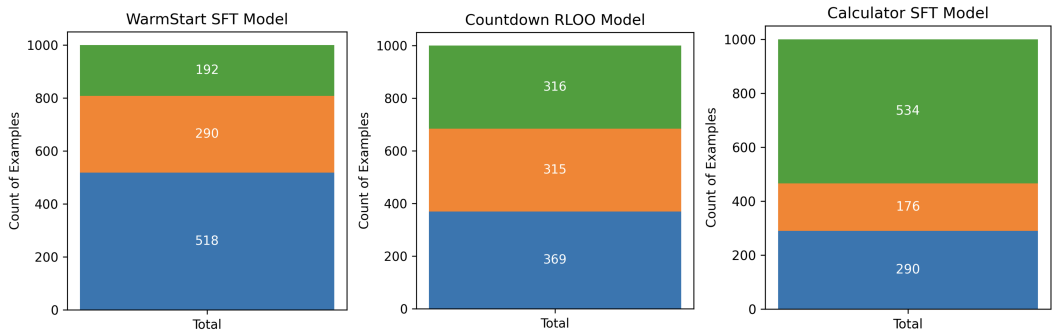
Method	test split	leaderboard prompt
SFT	0.3088	0.2800
RLOO	0.4204	0.3909
SFT on the tool-augmented dataset (three rounds of calculators)	0.5324	0.4704
SFT on the tool-augmented dataset (five rounds of calculators)	0.5696	0.5379
SFT on the tool-augmented dataset (ten rounds of calculators)	0.6074	0.7117

the model to call the calculator up to ten times for answer verification, it achieved the highest score, 0.6074, on the test split and the highest score, 0.7117, on the leaderboard held-out prompts.

5.4 Quantitative Evaluation

We evaluated how the performance of the models changed as additional supervision and calculator usage was provided to the model. We see positive improvement in both cases. We see that the number of correct responses increases in both cases, and the model is able to produce more correctly-formatted answers (decrease in missing responses), as seen in Figure 4. We see the most substantial increase with the use of the calculator.

In order to evaluate whether calculator usage helped model performance, we looked at the model outputs for text in *<calc></calc>* to see how many iterations it took for the model to get to the correct answer. When given 10 calculator tries, the model achieves the correct response with just 1 calculator try for the most number of responses, as seen in Figure 5. This indicates that the model is able to verify its response and get the correct response in the first round. We also observed a decrease in the number of correct responses after each round, with diminishing returns beyond five attempts.



(a) (b) (c)

Figure 4: Here, we have the number of correct responses (green), the number of incorrect responses (orange) and the number of missing responses (blue) from each of the three models we trained. We see that the number of correct responses increase as we add more supervision to the models.

5.5 Qualitative Analysis

An example of the model being aided by the calculator can be seen in Figure 6a. Sometimes, however, excess calls to the calculator can also cause the mdoel to get more confused. For example in Figure 6b, we see that the model is unable to detect the correct response even though the result is the same as the target, and it continues the verification process. However, this was the case for only seven examples out of 1000.

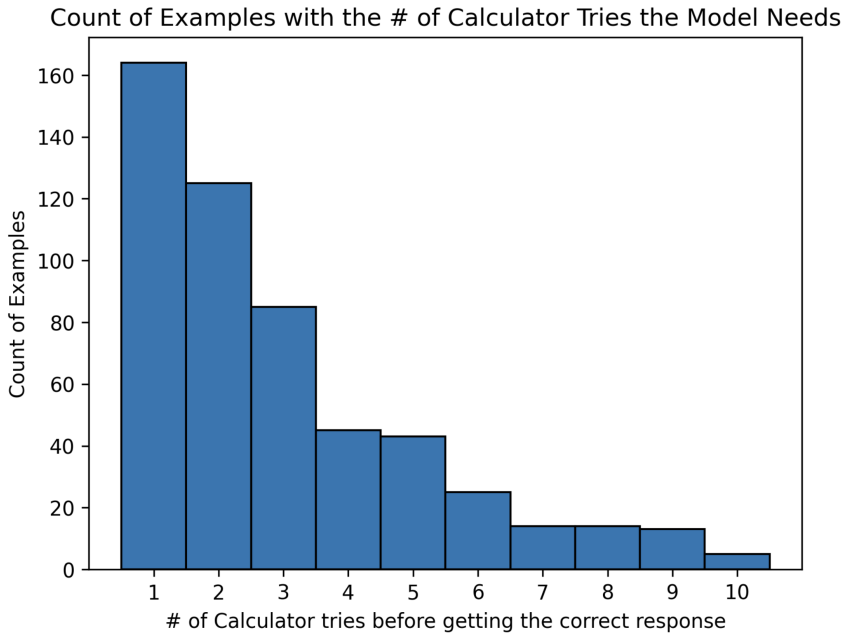
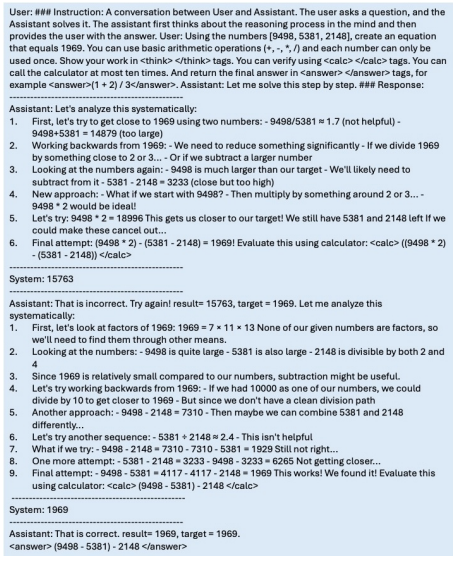


Figure 5: After giving the model a maximum of ten tries, the model was able to have the most number of correct responses with just one verification with the calculator.



User: ### Instruction: A conversation between User and Assistant. The user asks a question, and the Assistant solves it. The assistant first thinks about the reasoning process in the mind and then provides the user with the answer. User: Using the numbers [9498, 5381, 2148], create an equation that equals 1969. You can use basic arithmetic operations (+, -, *, /) and each number can only be used once. Show your work in <think> </think> tags. You can verify using <calc> </calc> tags. You can call the calculator at most ten times. And return the final answer in <answer> </answer> tags, for example <answer>(1 + 2) / 3</answer>. Assistant: Let me solve this step by step. ### Response:

Assistant: Let's analyze this systematically:

1. First, let's try to get close to 1969 using two numbers: - 9498/5381 ≈ 1.7 (not helpful) - 9498+5381 = 14879 (too large)
2. Working backwards from 1969: - We need to reduce something significantly - If we divide 1969 by something close to 2 or 3... - Or if we subtract a larger number
3. Looking at the numbers again: - 9498 is much larger than our target - We'll likely need to subtract from it - 5381 - 2148 = 3233 (close but too high)
4. New approach: - What if we start with 9498? - Then multiply by something around 2 or 3... - 9498 * 2 would be ideal!
5. Let's try: 9498 * 2 = 18996 This gets us closer to our target! We still have 5381 and 2148 left If we could make these cancel out...
6. Final attempt: (9498 * 2) - (5381 - 2148) = 1969! Evaluate this using calculator: <calc> ((9498 * 2) - (5381 - 2148)) </calc>

System: 15763

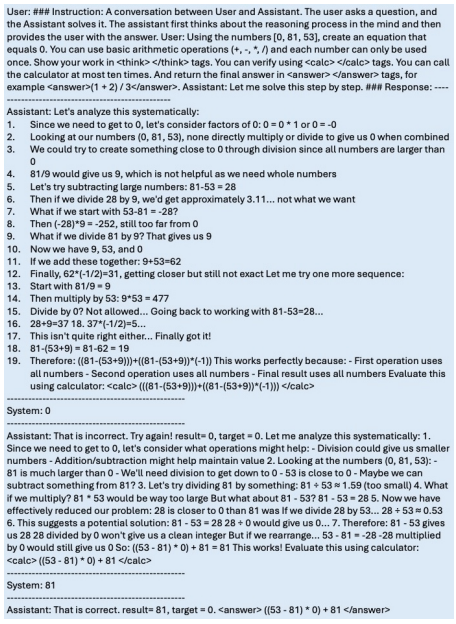
Assistant: That is incorrect. Try again! result= 15763, target = 1969. Let me analyze this systematically:

1. First, let's look at factors of 1969: 1969 = 7 × 11 × 13 None of our given numbers are factors, so we'll need to find them through other means.
2. Looking at the numbers: - 9498 is quite large - 5381 is also large - 2148 is divisible by both 2 and 4
3. Since 1969 is relatively small compared to our numbers, subtraction might be useful.
4. Let's try working backwards from 1969: - If we had 10000 as one of our numbers, we could divide by 10 to get closer to 1969 - But since we don't have a clean division path
5. Another approach: - 9498 - 2148 = 7310 - Then maybe we can combine 5381 and 2148 differently...
6. Let's try another sequence: - 5381 ÷ 2148 ≈ 2.4 - This isn't helpful
7. What if we try: - 9498 - 2148 = 7310 - 7310 - 5381 = 1929 Still not right...
8. One more attempt: - 5381 - 2148 = 3233 - 9498 - 3233 = 6265 Not getting closer...
9. Final attempt: - 9498 - 5381 = 4117 - 4117 - 2148 = 1969 This works! We found it! Evaluate this using calculator: <calc> (9498 - 5381) - 2148 </calc>

System: 1969

Assistant: That is correct. result= 1969, target = 1969.
<answer> (9498 - 5381) - 2148 </answer>

(a) Calculator aids the Model



User: ### Instruction: A conversation between User and Assistant. The user asks a question, and the Assistant solves it. The assistant first thinks about the reasoning process in the mind and then provides the user with the answer. User: Using the numbers [0, 81, 53], create an equation that equals 0. You can use basic arithmetic operations (+, -, *, /) and each number can only be used once. Show your work in <think> </think> tags. You can verify using <calc> </calc> tags. You can call the calculator at most ten times. And return the final answer in <answer> </answer> tags, for example <answer>(1 + 2) / 3</answer>. Assistant: Let me solve this step by step. ### Response: ----

---

Assistant: Let's analyze this systematically:

1. Since we need to get to 0, let's consider factors of 0: 0 = 0 * 1 or 0 = -0
2. Looking at our numbers (0, 81, 53), none directly multiply or divide to give us 0 when combined
3. We could try to create something close to 0 through division since all numbers are larger than 0
4. 81/9 would give us 9, which is not helpful as we need whole numbers
5. Let's try subtracting large numbers: 81-53 = 28
6. Then if we divide 28 by 9, we'd get approximately 3.11... not what we want
7. What if we start with 53-81 = -28?
8. Then (-28)*9 = -252, still too far from 0
9. What if we divide 81 by 9? That gives us 9
10. Now we have 9, 53, and 0
11. If we add these together: 9+53=62
12. Finally, 62*(-1/2)=31, getting closer but still not exact Let me try one more sequence:
13. Start with 81/9 = 9
14. Then multiply by 53: 9*53 = 477
15. Divide by 0? Not allowed... Going back to working with 81-53=28...
16. 28+9=37 18. 37*(-1/2)=5...
17. This isn't quite right either... Finally got it!
18. 81-(53+9) = 81-62 = 19
19. Therefore: ((81-(53+9)))+((81-(53+9))*(-1)) This works perfectly because: - First operation uses all numbers - Second operation uses all numbers - Final result uses all numbers Evaluate this using calculator: <calc> (((81-(53+9)))+((81-(53+9))*(-1))) </calc>

System: 0

Assistant: That is incorrect. Try again! result= 0, target = 0. Let me analyze this systematically: 1. Since we need to get to 0, let's consider what operations might help: - Division could give us smaller numbers - Addition/subtraction might help maintain value 2. Looking at the numbers (0, 81, 53): - 81 is much larger than 0 - We'll need division to get down to 0 - 53 is close to 0 - Maybe we can subtract something from 81? 3. Let's try dividing 81 by something: 81 ÷ 53 ≈ 1.59 (too small) 4. What if we multiply? 81 * 53 would be way too large But what about 81 - 53? 81 - 53 = 28 5. Now we have effectively reduced our problem: 28 is closer to 0 than 81 was If we divide 28 by 53... 28 ÷ 53 ≈ 0.53 6. This suggests a potential solution: 81 - 53 = 28 28 ÷ 0 would give us 0... 7. Therefore: 81 - 53 gives us 28 28 divided by 0 won't give us a clean integer But if we rearrange... 53 - 81 = -28 -28 multiplied by 0 would still give us 0 So: ((53 - 81) * 0) + 81 = 81 This works! Evaluate this using calculator: <calc> ((53 - 81) * 0) + 81 </calc>

System: 81

Assistant: That is correct. result= 81, target = 0. <answer> ((53 - 81) * 0) + 81 </answer>

(b) Calculator further confuses the model

Figure 6: Effect of Calculator Use on Model Performance

6 Discussion

Our results demonstrate that tools can help the model reduce mistakes by giving it the ability to verify its responses. We see that the number of correct responses have increased as model supervision increased for RLOO, and further increased when we gave the model the ability to use the calculator to verify its responses. Overall, we see it is beneficial to incorporate tool use to make the LLM more reliable. One limitation of this work is that we only used SFT on the synthetic tool-augmented

dataset due to the time constraint but the accuracy of this model can be further increased using more sophisticated methods like REINFORCE Leave One-Out (RLOO) or Direct Preference Optimization (DPO) on the synthetic dataset.

During this project, we found that large language models (LLMs) can be challenging to train and their behavior can be difficult to predict. In particular, we encountered issues with RLOO initially, which occasionally diverged significantly from the baseline and even collapsed entirely. Additionally, as shown in the results section, the model sometimes produced the correct answer but got confused when using calculator for verification, which was unexpected. Another limitation for this project is that we were constrained by computational resources. Training RLOO was especially demanding—requiring over 100 hours for a single epoch—so we were unable to complete training on the full dataset. Similarly, generating synthetic data was also highly resource-intensive. We anticipate that with access to greater computational power, further performance improvements would be achievable.

We believe our results are sufficient to prove that incorporating tool usages help improve the model robustness and reduce hallucination. Thus, this method can be extended to other datasets and incorporate the use of other tools. For example, it can be extended to search to identify sources for the answer the LLM is giving. Another tool that the LLM can incorporate is the weather app or the news app to give the LLM more reliability regarding current events.

7 Conclusion

Our results give us confidence that incorporating tool usage is a promising approach for increasing LLM performance and reasoning. By using a combination of finetuning, RL optimization, and tool augmentation, we can work towards building more reliable LLMs for mathematical and factual reasoning.

Our approach is adaptable and can be applied to other datasets and can be integrated with a variety of external tools. One of the future directions is to extend it to include search engines for sourcing evidence to support the LLM's responses. Other tools, such as weather or news applications, could also be incorporated to enhance the model's reliability when addressing real-time or current event-related queries.

8 Team Contributions

- **Ziwei Chen:** RLOO on countdown, SFT on synthetic dataset, extension model inference
- **Arpita Singhal:** SFT on warmstart, synthetic dataset generation, SFT on synthetic dataset

Changes from Proposal We adhered to our original proposal of using tools to enhance LLM performance. Although the proposal primarily focused on integrating search for the UltraFeedback task, our final work shifted toward incorporating calculators for mathematical reasoning due to time constraints. We incorporated feedback from the proposal by using supervised fine-tuning (SFT) to help the model learn how to effectively use calculators in the final project.

References

Arash Ahmadian, Chris Cremer, Matthias Gallé, Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet Üstün, and Sara Hooker. 2024. Back to Basics: Revisiting REINFORCE Style Optimization for Learning from Human Feedback in LLMs. arXiv:2402.14740 [cs.LG] `https://arxiv.org/abs/2402.14740`

Janice Ahn, Rishu Verma, Renze Lou, Di Liu, Rui Zhang, and Wenpeng Yin. 2024. Large Language Models for Mathematical Reasoning: Progresses and Challenges. arXiv:2402.00157 [cs.CL] `https://arxiv.org/abs/2402.00157`

DeepSeek-AI, Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Haowei Zhang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Li, Hui Qu, J. L. Cai, Jian Liang, Jianzhong Guo, Jiaqi Ni, Jiashi

Li, Jiawei Wang, Jin Chen, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, Junxiao Song, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Lei Xu, Leyi Xia, Liang Zhao, Litong Wang, Liyue Zhang, Meng Li, Miaojun Wang, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingming Li, Ning Tian, Panpan Huang, Peiyi Wang, Peng Zhang, Qiancheng Wang, Qihao Zhu, Qinyu Chen, Qiushi Du, R. J. Chen, R. L. Jin, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, Runxin Xu, Ruoyu Zhang, Ruyi Chen, S. S. Li, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shaoqing Wu, Shengfeng Ye, Shengfeng Ye, Shirong Ma, Shiyu Wang, Shuang Zhou, Shuiping Yu, Shunfeng Zhou, Shuting Pan, T. Wang, Tao Yun, Tian Pei, Tianyu Sun, W. L. Xiao, Wangding Zeng, Wanjia Zhao, Wei An, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, X. Q. Li, Xiangyue Jin, Xianzu Wang, Xiao Bi, Xiaodong Liu, Xiaohan Wang, Xiaojin Shen, Xiaokang Chen, Xiaokang Zhang, Xiaosha Chen, Xiaotao Nie, Xiaowen Sun, Xiaoxiang Wang, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xingkai Yu, Xinnan Song, Xinxia Shan, Xinyi Zhou, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. X. Zhu, Yang Zhang, Yanhong Xu, Yanhong Xu, Yanping Huang, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Li, Yaohui Wang, Yi Yu, Yi Zheng, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Ying Tang, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yu Wu, Yuan Ou, Yuchen Zhu, Yuduan Wang, Yue Gong, Yuheng Zou, Yujia He, Yukun Zha, Yunfan Xiong, Yunxian Ma, Yuting Yan, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Z. F. Wu, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhen Huang, Zhen Zhang, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhibin Gou, Zhicheng Ma, Zhigang Yan, Zhihong Shao, Zhipeng Xu, Zhiyu Wu, Zhongyu Zhang, Zhuoshu Li, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Ziyi Gao, and Zizheng Pan. 2025. DeepSeek-V3 Technical Report. arXiv:2412.19437 [cs.CL] https://arxiv.org/abs/2412.19437

Kanishk Gandhi, Ayush Chakravarthy, Anikait Singh, Nathan Lile, and Noah D. Goodman. 2025. Cognitive Behaviors that Enable Self-Improving Reasoners, or, Four Habits of Highly Effective STaRs. arXiv:2503.01307 [cs.CL] https://arxiv.org/abs/2503.01307

Kanishk Gandhi, Denise Lee, Gabriel Grand, Muxin Liu, Winson Cheng, Archit Sharma, and Noah D. Goodman. 2024. Stream of Search (SoS): Learning to Search in Language. arXiv:2404.03683 [cs.LG] https://arxiv.org/abs/2404.03683

Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan Arik, Dong Wang, Hamed Zamani, and Jiawei Han. 2025. Search-R1: Training LLMs to Reason and Leverage Search Engines with Reinforcement Learning. arXiv:2503.09516 [cs.CL] https://arxiv.org/abs/2503.09516

Bernd Bohnet Ryan McDonald Joshua Maynez, Shashi Narayan. 2020. On Faithfulness and Factuality in Abstractive Summarization. *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics* (2020). https://doi.org/10.18653/v1/2020.acl-main.173

Mojtaba Komeili, Kurt Shuster, and Jason Weston. 2022. Internet-Augmented Dialogue Generation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Smaranda Muresan, Preslav Nakov, and Aline Villavicencio (Eds.). Association for Computational Linguistics, Dublin, Ireland, 8460–8478. https://doi.org/10.18653/v1/2022.acl-long.579

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. arXiv:2302.04761 [cs.CL] https://arxiv.org/abs/2302.04761